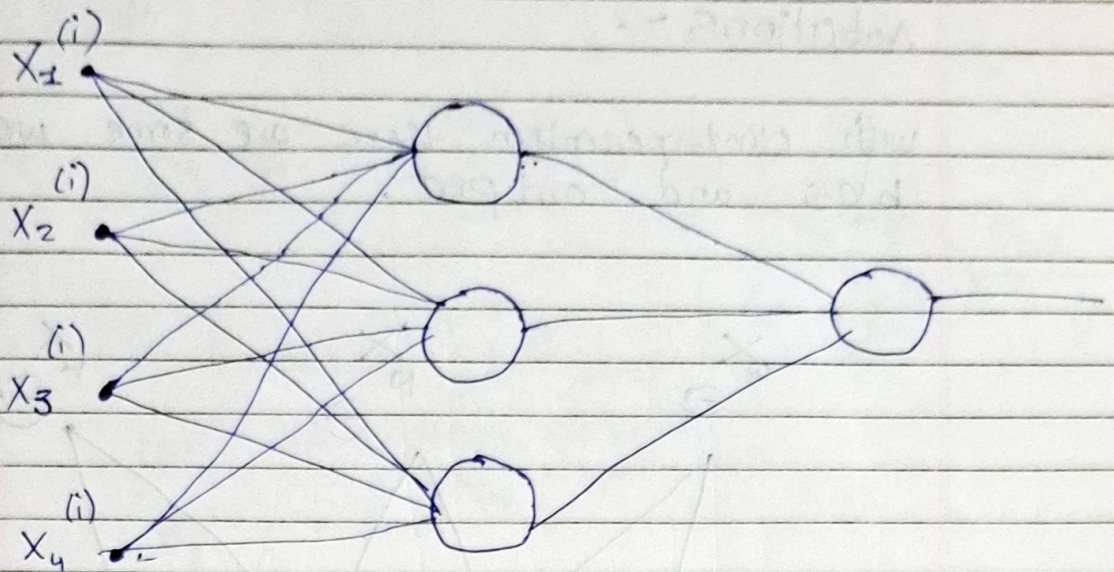


Artificial Neural Networks (ANNs)



Input
Layer

Hidden
Layer-I

~~Output~~
Layer

(Layer I)

(Layer II)

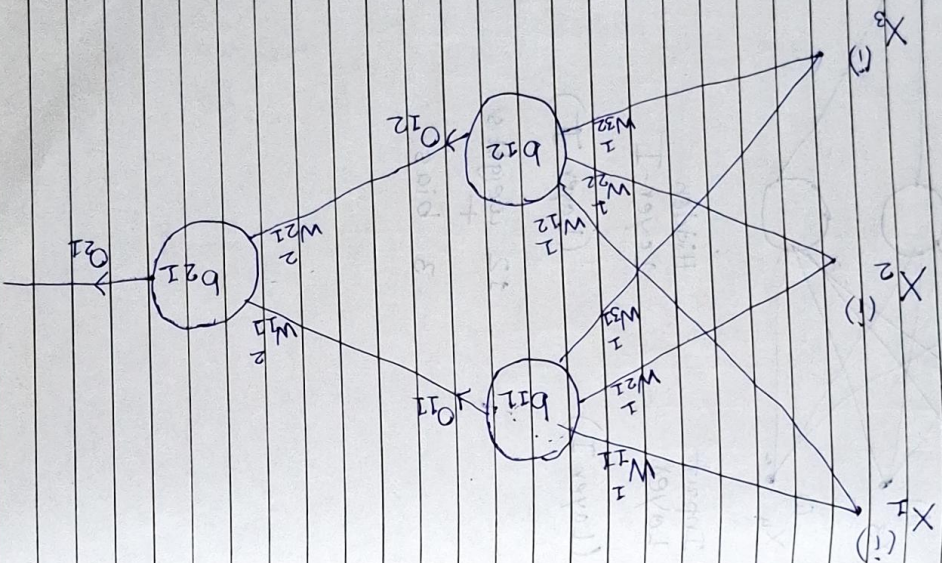
(Layer III)

12 weights
+
3 bias

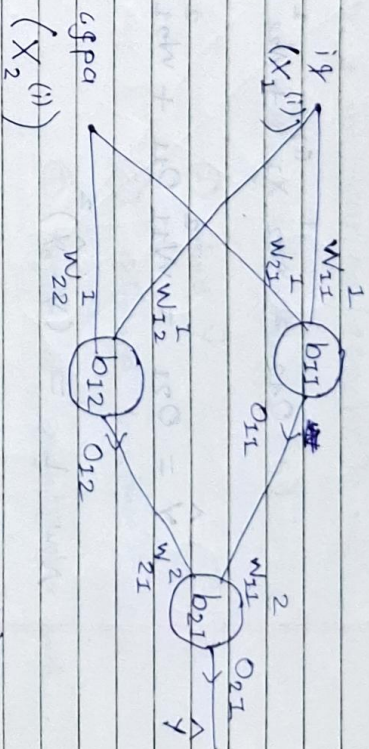
3 weights
+
2 bias

Notations --

with every perception there are some weights, bias and output.



Back Propagation --



(linear, sigmoid, softmax, relu)

- there are activation function attached with all the perceptrons -

- for the model we have to select loss function.

(mse, mae, rmse, ~~log~~ binary cross entropy, categorical cross entropy, sparse categorical cross entropy.)

- let's take activation function for all the perceptron as "linear". and taking loss function as "mse".

$$O_{11} = w_{11}^1 x_1^{(1)} + w_{01}^1 x_0^{(1)} + b_{11}$$

$$O_{12} = w_{12}^1 x_1^{(1)} + w_{02}^1 x_0^{(1)} + b_{12}$$

$$\hat{y} = O_{21} = w_{11}^2 O_{11} + w_{21}^2 O_{12} + b_{21}$$

$$\text{Now, } L = (y - \hat{y})^2$$

- we will use gradient descent for training the data.

- we have to find

$$\frac{\partial L}{\partial w_{12}}, \frac{\partial L}{\partial w_{21}}, \frac{\partial L}{\partial b_{21}}$$

$$\frac{\partial L}{\partial w_{11}}, \frac{\partial L}{\partial w_{21}}, \frac{\partial L}{\partial b_{21}}$$

$$\frac{\partial L}{\partial w_{12}}, \frac{\partial L}{\partial w_{22}}, \frac{\partial L}{\partial b_{22}}$$

- after finding this we will use this to update weights and bias.

for j in range (epochs):

for j in range (n):

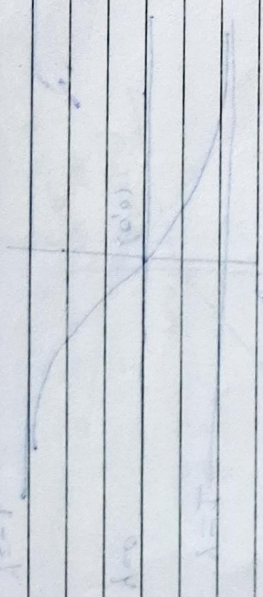
① find $\hat{y}$

② calculate loss (mse)

③ Adjust all weights and bias

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w_{old}}$$

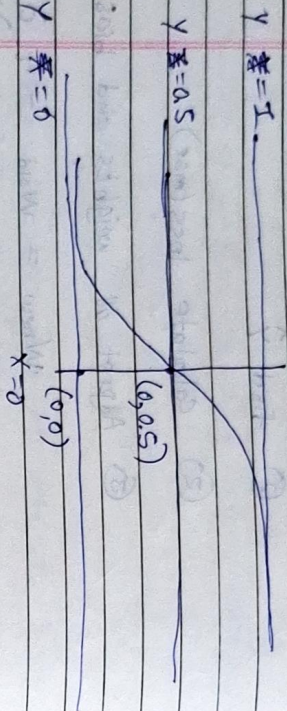
$$s = g + 1$$



$$y - \hat{y} = (5) \text{ and } = (5) ?$$

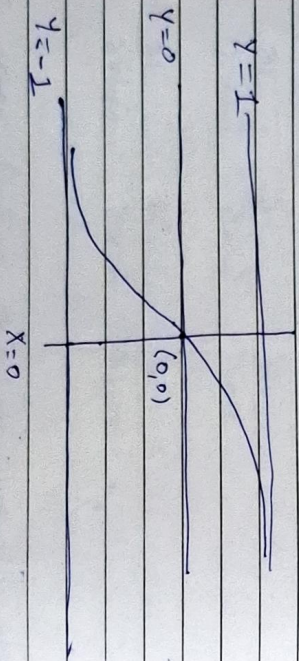
→ Activation functions :
(Sigmoid, relu and tanh)

Sigmoid Activation function :



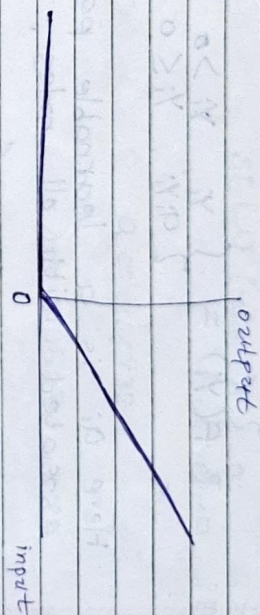
$$f(z) = \frac{1}{1 + e^{-z}}$$

Tanh or Hyperbolic Tangent Activation function



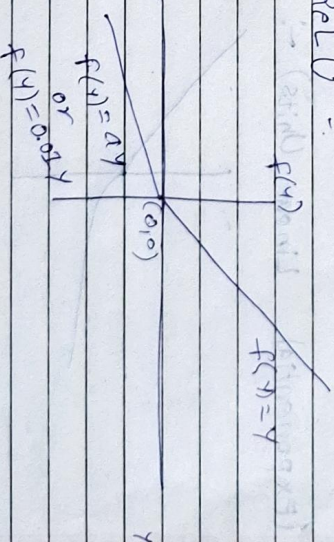
$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \frac{e^{2z} - 1}{e^{2z} + 1}$$

ReLU (Rectified Linear Unit) Activation function :



$$f(z) = \begin{cases} z & , z > 0 \\ 0 & , z < 0 \end{cases}$$

Leaky ReLU :



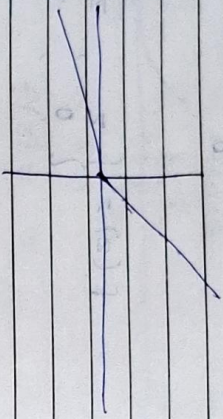
$$f(y_i) = \begin{cases} y_i & , \text{if } y_i > 0 \\ 0 & , \text{if } y_i \leq 0 \end{cases}$$

α_i 's can be 0.01 or something like that.
(and it is fixed)

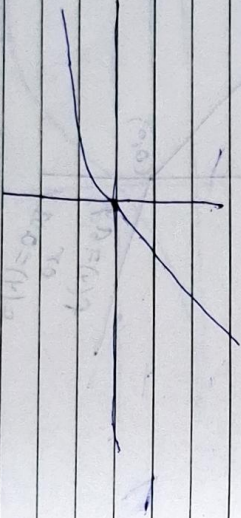
Parametric ReLU :-

$$f(y_i) = \begin{cases} y_i, & y_i > 0 \\ a_i y_i, & y_i \leq 0 \end{cases}$$

Here a_i is a learnable parameter associated with all nodes.



(Exponential Linear Units) :-



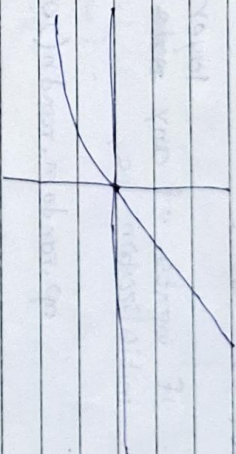
$$f(x) = \text{ELU}(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

Selu (Scaled Exponential Linear Unit) :-

$$\text{SELU}(x) = \lambda \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & \text{if } x \leq 0 \end{cases}$$

$$\alpha = 1.6732$$

$$\lambda = 1.0507$$



For Weight Initializations :-

Xavier (Normal) :-
(Glorot)

- Random initialization on Normal Distribution with mean 0 and standard deviation $\sqrt{\frac{1}{fan_in(inputs)}}$.

ex. if inputs of any ~~node~~ ^{layer} is 10 then initialization is,
(for each node)

$$np.random.randn(10, x) * \sqrt{\frac{1}{10}}$$

x is no. of nodes

He (Normal) :-

- Random initialization on Normal distribution with mean 0 and standard deviation

$$\sqrt{\frac{2}{fan_in}}$$

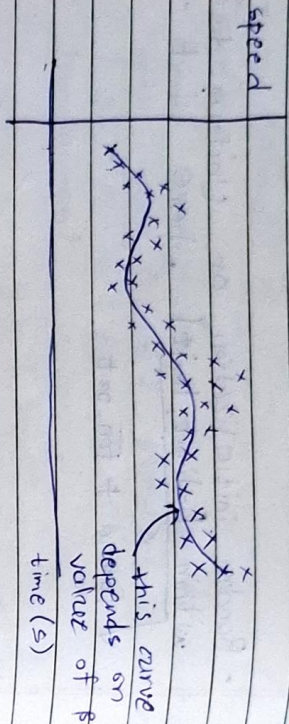
Xavier (Uniform) :-
(Glorot)

- Random initialization on Uniform distribution within $[-limit, limit]$ where limit is $\sqrt{\frac{6}{fan_in + fan_out}}$

He (Uniform) :-

- Random initialization on Uniform distribution within $[-limit, limit]$ where limit is $\sqrt{\frac{6}{fan_in}}$

Exponentially weighted moving average:



$$V_t = \beta V_{t-1} + (1-\beta)O_t$$

time(s)	speed (m/s)
1	25
2	27
3	23
4	29
5	32
6	28
7	29
8	32

$$0 < \beta < 1$$

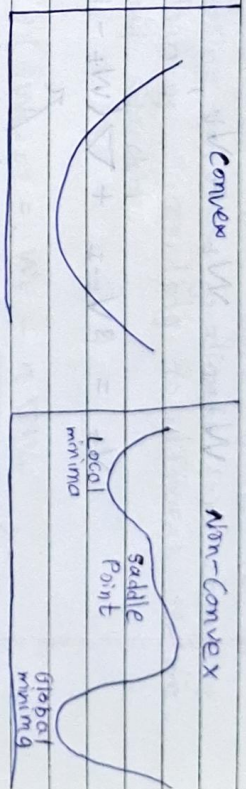
$$V_0 = O_0$$

- As β increases we get more smooth or mature graph.

- As β decrease we get moody and more fluctuated graph.

SGD with Momentum

There are some problems with non-convex optimization



- ① Local minimas
- ② Saddle points
- ③ High curvature.

- We are kind of using EWMA in this optimization technique.

- For every iteration of updating parameters we are using previous gradient as well for momentum.

$$W_{t+1} = W_t - \Delta W_t$$

$$V_t = \beta * V_{t-1} + \eta \Delta W_t$$

$$0 < \beta < 1$$

Nesterov Accelerated Gradient

- It has some changes from momentum

$$W_{t+1} = W_t - V_t,$$

$$V_t = \beta V_{t-1} + \nabla (W_t - \beta V_{t-1})$$

- For finding current gradient term we are going one step forward to see what will happen in future, and by that result we are calculating gradient to better convergence.

AdaGrad (Adaptive Gradient)

- It is also using concept of previous gradients, but this time it is using previous gradients for changing learning according to different features and situation.

$$W_{t+1} = W_t - \frac{\eta \nabla W_t}{\sqrt{V_t + \epsilon}},$$

$$\sqrt{V_t + \epsilon}$$

small number
(used when $V_t = 0$)

$$V_t = V_{t-1} + (\nabla W_t)^2$$

- So, V_t is sum of gradients.
- this optimizer is used for sparse data or sparse features.
- sparse features have low gradients, so learning rate would be high, and it will gain gradient from that.
- for non-sparse features, V_t would be high, so there is some problems are there in AdaGrad.

RMSProp (Root mean square propagation)

- it has some similarity with Adagrad.

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t,$$

$$v_t = \beta v_{t-1} + (1-\beta) (\nabla w_t)^2,$$

$$0 < \beta < 1$$

- Now, we have tackled that problem, when v_t becomes very high for non-sparse features.

- This optimizer is one of the best, because it is solving lots of problem which we have seen before.

Adam (Adaptive Moment Estimation)

- Adam is using concepts of Momentum and Adagrad together.

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} * m_t,$$

$$\text{(momentum)} \quad m_t = \beta_1 m_{t-1} + (1-\beta_1) \nabla w_t$$

$$\text{(Adagrad)} \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) (\nabla w_t)^2$$

$$\beta_1, \beta_2 \in (0, 1)$$

- Adam is also best algorithm, which is used in ~~a~~ deep learning most of the time.

CNN

(Convolutional neural network)

- CNNs are widely used in areas such as image classification, object detection, facial recognition, and medical image analysis. They can be quite effective for classifying non-image data such as audio, time series, and signal data.
- Structure of CNN involves convolutional layers, flatten layer, fully connected (dense) layers.
- Convolution is the process of applying a filter ~~a~~ to input data in a convolutional neural network (CNN) to produce a feature map. The feature map highlights the locations and strengths of detected features in the image.
- A Convolutional Filter is a small matrix used in Convolutional Neural Networks (CNNs) to extract features from images by sliding over the image and combining with the underlying pixels to obtain the maximum value from each convolution.

- feature maps are the output produced after applying the convolution operation with different filters. Each feature map corresponds to a specific learned feature and multiple feature maps together represent important characteristics of the input image.

Padding :

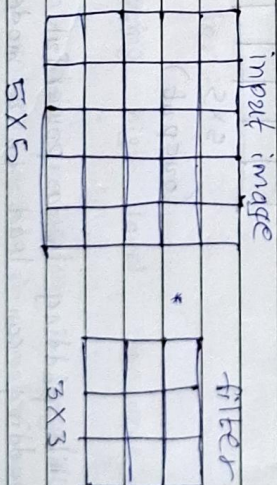
- Padding is a technique used in convolutional neural networks (CNNs) to add extra pixels or values around the input data.
- it's important for a number of reasons :
 - prevents data loss at the edges.
 - controlling output size.
 - making training more stable
 - padding helps the kernel visit pixels around the corners more times to extract important features.
- Making CNN more effective

types of Padding :

- Zero padding : Adds zeros (or any constant value) to the edges of the data.
- Valid Padding : Also known as "no padding", this involves not adding any extra elements to the data.
- Some padding : Padding is applied and the output image size should be equal to the input image size.

example :

normal case :



output image

3x3

- model capacity: larger strides may cause the model to lose some detailed information because the filter skips over some pixels. This could reduce model accuracy.

- Shape of input image: $n \times n$
 shape of filter: $K \times K$

padding added is: p
 value of stride: s

then, shape of output image:

$$\left(\frac{1 + n + 2 * p - k}{s} \right) \times \left(\frac{1 + n + 2 * p - k}{s} \right)$$

Pooling layer:

- In CNN, the output features maps are from the convolutional layers are downsampled by using pooling layers. The basic goal of pooling is to maintain the most relevant information while decreasing the inputs spatial size. It is reducing or controlling overfitting, and extracting dominant features.

Note:

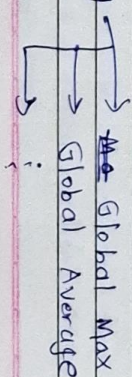
Translational invariance refers to the property of a system or model where the behavior or performance is unchanged even if its input is shifted or translated.

- Pooling techniques like Max pooling provides Translation invariance to the model.

Types of Pooling:

- Max Pooling
- Min Pooling
- Avg Pooling
- 1-2 Pooling

Global Pooling



example:

Max pooling

3	1	1	3
2	5	0	2
1	4	2	1
4	7	2	4

kernel-size = (2, 2), strides = 2,
padding = 'valid'

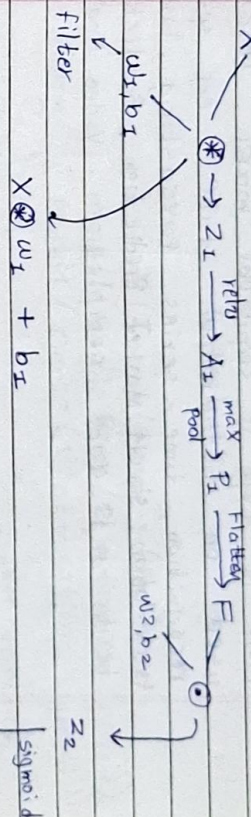
Max Pooling

5	3
7	4

Global Max Pooling

- For backpropagation in CNN, you can refer to the campus or medium articles. I am just going to give you brief intro in it.

(input image)



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial F} \cdot \frac{\partial F}{\partial F_1} \cdot \frac{\partial F_1}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial w_1}$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial F} \cdot \frac{\partial F}{\partial F_1} \cdot \frac{\partial F_1}{\partial A_1} \cdot \frac{\partial A_1}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial b_1}$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial w_2}$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial b_2}$$

In further case we have to find all these gradients and apply gradient descent on all four parameters.

RNN (Recurrent NN)

RNN is type of sequential model which is used on sequential data. eg. speech recognition, time-series forecasting, language translation, stock market prediction, predictive text and auto completion, Video Analysis.

eg. Hi my name is viraal

first we have to encode this data into numbers, we can use one hot encoding, label encoding or something else to encode the data.

let's say we are using label encoding.

then ~~the~~ above eg. will be,

[1, 2, 3, 4, 5]

t=1 t=2 t=3 t=4 t=5

now we can pass ~~a~~ word by word with sequence to the RNN.

If we have used label encoding then it will be,

$X_{i1} = [1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1]$

$X_{i2} \quad X_{i3} \quad X_{i4} \quad X_{i5}$

t=1 t=2 t=3 t=4 t=5

If we take more ex. then unique words will increase. (X_{it})

- Assume we are using below model,

$m = \text{sequential}()$

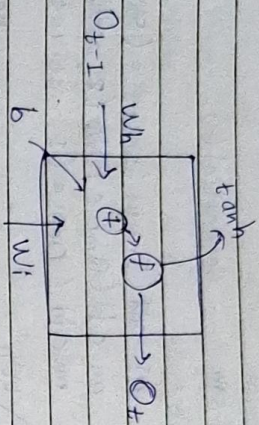
$m.add(\text{Input}(\text{shape} = (5, 5)))$

$m.add(\text{Simple RNN}(2, \text{activation} = 'relu'))$

(random) $m.add(\text{Dense}(1, \text{activation} = 'sigmoid'))$

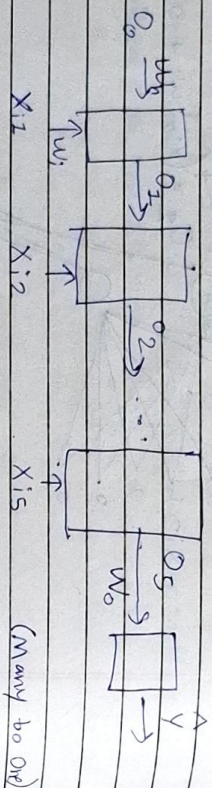
W_h O_1 O_2 O_3 O_4 O_5 O_6 O_7 O_8 O_9 O_{10} O_{11} O_{12} O_{13} O_{14} O_{15} O_{16} O_{17} O_{18} O_{19} O_{20} O_{21} O_{22} O_{23} O_{24} O_{25} O_{26} O_{27} O_{28} O_{29} O_{30} O_{31} O_{32} O_{33} O_{34} O_{35} O_{36} O_{37} O_{38} O_{39} O_{40} O_{41} O_{42} O_{43} O_{44} O_{45} O_{46} O_{47} O_{48} O_{49} O_{50} O_{51} O_{52} O_{53} O_{54} O_{55} O_{56} O_{57} O_{58} O_{59} O_{60} O_{61} O_{62} O_{63} O_{64} O_{65} O_{66} O_{67} O_{68} O_{69} O_{70} O_{71} O_{72} O_{73} O_{74} O_{75} O_{76} O_{77} O_{78} O_{79} O_{80} O_{81} O_{82} O_{83} O_{84} O_{85} O_{86} O_{87} O_{88} O_{89} O_{90} O_{91} O_{92} O_{93} O_{94} O_{95} O_{96} O_{97} O_{98} O_{99} O_{100} O_{101} O_{102} O_{103} O_{104} O_{105} O_{106} O_{107} O_{108} O_{109} O_{110} O_{111} O_{112} O_{113} O_{114} O_{115} O_{116} O_{117} O_{118} O_{119} O_{120} O_{121} O_{122} O_{123} O_{124} O_{125} O_{126} O_{127} O_{128} O_{129} O_{130} O_{131} O_{132} O_{133} O_{134} O_{135} O_{136} O_{137} O_{138} O_{139} O_{140} O_{141} O_{142} O_{143} O_{144} O_{145} O_{146} O_{147} O_{148} O_{149} O_{150} O_{151} O_{152} O_{153} O_{154} O_{155} O_{156} O_{157} O_{158} O_{159} O_{160} O_{161} O_{162} O_{163} O_{164} O_{165} O_{166} O_{167} O_{168} O_{169} O_{170} O_{171} O_{172} O_{173} O_{174} O_{175} O_{176} O_{177} O_{178} O_{179} O_{180} O_{181} O_{182} O_{183} O_{184} O_{185} O_{186} O_{187} O_{188} O_{189} O_{190} O_{191} O_{192} O_{193} O_{194} O_{195} O_{196} O_{197} O_{198} O_{199} O_{200} O_{201} O_{202} O_{203} O_{204} O_{205} O_{206} O_{207} O_{208} O_{209} O_{210} O_{211} O_{212} O_{213} O_{214} O_{215} O_{216} O_{217} O_{218} O_{219} O_{220} O_{221} O_{222} O_{223} O_{224} O_{225} O_{226} O_{227} O_{228} O_{229} O_{230} O_{231} O_{232} O_{233} O_{234} O_{235} O_{236} O_{237} O_{238} O_{239} O_{240} O_{241} O_{242} O_{243} O_{244} O_{245} O_{246} O_{247} O_{248} O_{249} O_{250} O_{251} O_{252} O_{253} O_{254} O_{255} O_{256} O_{257} O_{258} O_{259} O_{260} O_{261} O_{262} O_{263} O_{264} O_{265} O_{266} O_{267} O_{268} O_{269} O_{270} O_{271} O_{272} O_{273} O_{274} O_{275} O_{276} O_{277} O_{278} O_{279} O_{280} O_{281} O_{282} O_{283} O_{284} O_{285} O_{286} O_{287} O_{288} O_{289} O_{290} O_{291} O_{292} O_{293} O_{294} O_{295} O_{296} O_{297} O_{298} O_{299} O_{300} O_{301} O_{302} O_{303} O_{304} O_{305} O_{306} O_{307} O_{308} O_{309} O_{310} O_{311} O_{312} O_{313} O_{314} O_{315} O_{316} O_{317} O_{318} O_{319} O_{320} O_{321} O_{322} O_{323} O_{324} O_{325} O_{326} O_{327} O_{328} O_{329} O_{330} O_{331} O_{332} O_{333} O_{334} O_{335} O_{336} O_{337} O_{338} O_{339} O_{340} O_{341} O_{342} O_{343} O_{344} O_{345} O_{346} O_{347} O_{348} O_{349} O_{350} O_{351} O_{352} O_{353} O_{354} O_{355} O_{356} O_{357} O_{358} O_{359} O_{360} O_{361} O_{362} O_{363} O_{364} O_{365} O_{366} O_{367} O_{368} O_{369} O_{370} O_{371} O_{372} O_{373} O_{374} O_{375} O_{376} O_{377} O_{378} O_{379} O_{380} O_{381} O_{382} O_{383} O_{384} O_{385} O_{386} O_{387} O_{388} O_{389} O_{390} O_{391} O_{392} O_{393} O_{394} O_{395} O_{396} O_{397} O_{398} O_{399} O_{400} O_{401} O_{402} O_{403} O_{404} O_{405} O_{406} O_{407} O_{408} O_{409} O_{410} O_{411} O_{412} O_{413} O_{414} O_{415} O_{416} O_{417} O_{418} O_{419} O_{420} O_{421} O_{422} O_{423} O_{424} O_{425} O_{426} O_{427} O_{428} O_{429} O_{430} O_{431} O_{432} O_{433} O_{434} O_{435} O_{436} O_{437} O_{438} O_{439} O_{440} O_{441} O_{442} O_{443} O_{444} O_{445} O_{446} O_{447} O_{448} O_{449} O_{450} O_{451} O_{452} O_{453} O_{454} O_{455} O_{456} O_{457} O_{458} O_{459} O_{460} O_{461} O_{462} O_{463} O_{464} O_{465} O_{466} O_{467} O_{468} O_{469} O_{470} O_{471} O_{472} O_{473} O_{474} O_{475} O_{476} O_{477} O_{478} O_{479} O_{480} O_{481} O_{482} O_{483} O_{484} O_{485} O_{486} O_{487} O_{488} O_{489} O_{490} O_{491} O_{492} O_{493} O_{494} O_{495} O_{496} O_{497} O_{498} O_{499} O_{500} O_{501} O_{502} O_{503} O_{504} O_{505} O_{506} O_{507} O_{508} O_{509} O_{510} O_{511} O_{512} O_{513} O_{514} O_{515} O_{516} O_{517} O_{518} O_{519} O_{520} O_{521} O_{522} O_{523} O_{524} O_{525} O_{526} O_{527} O_{528} O_{529} O_{530} O_{531} O_{532} O_{533} O_{534} O_{535} O_{536} O_{537} O_{538} O_{539} O_{540} O_{541} O_{542} O_{543} O_{544} O_{545} O_{546} O_{547} O_{548} O_{549} O_{550} O_{551} O_{552} O_{553} O_{554} O_{555} O_{556} O_{557} O_{558} O_{559} O_{560} O_{561} O_{562} O_{563} O_{564} O_{565} O_{566} O_{567} O_{568} O_{569} O_{570} O_{571} O_{572} O_{573} O_{574} O_{575} O_{576} O_{577} O_{578} O_{579} O_{580} O_{581} O_{582} O_{583} O_{584} O_{585} O_{586} O_{587} O_{588} O_{589} O_{590} O_{591} O_{592} O_{593} O_{594} O_{595} O_{596} O_{597} O_{598} O_{599} O_{600} O_{601} O_{602} O_{603} O_{604} O_{605} O_{606} O_{607} O_{608} O_{609} O_{610} O_{611} O_{612} O_{613} O_{614} O_{615} O_{616} O_{617} O_{618} O_{619} O_{620} O_{621} O_{622} O_{623} O_{624} O_{625} O_{626} O_{627} O_{628} O_{629} O_{630} O_{631} O_{632} O_{633} O_{634} O_{635} O_{636} O_{637} O_{638} O_{639} O_{640} O_{641} O_{642} O_{643} O_{644} O_{645} O_{646} O_{647} O_{648} O_{649} O_{650} O_{651} O_{652} O_{653} O_{654} O_{655} O_{656} O_{657} O_{658} O_{659} O_{660} O_{661} O_{662} O_{663} O_{664} O_{665} O_{666} O_{667} O_{668} O_{669} O_{670} O_{671} O_{672} O_{673} O_{674} O_{675} O_{676} O_{677} O_{678} O_{679} O_{680} O_{681} O_{682} O_{683} O_{684} O_{685} O_{686} O_{687} O_{688} O_{689} O_{690} O_{691} O_{692} O_{693} O_{694} O_{695} O_{696} O_{697} O_{698} O_{699} O_{700} O_{701} O_{702} O_{703} O_{704} O_{705} O_{706} O_{707} O_{708} O_{709} O_{710} O_{711} O_{712} O_{713} O_{714} O_{715} O_{716} O_{717} O_{718} O_{719} O_{720} O_{721} O_{722} O_{723} O_{724} O_{725} O_{726} O_{727} O_{728} O_{729} O_{730} O_{731} O_{732} O_{733} O_{734} O_{735} O_{736} O_{737} O_{738} O_{739} O_{740} O_{741} O_{742} O_{743} O_{744} O_{745} O_{746} O_{747} O_{748} O_{749} O_{750} O_{751} O_{752} O_{753} O_{754} O_{755} O_{756} O_{757} O_{758} O_{759} O_{760} O_{761} O_{762} O_{763} O_{764} O_{765} O_{766} O_{767} O_{768} O_{769} O_{770} O_{771} O_{772} O_{773} O_{774} O_{775} O_{776} O_{777} O_{778} O_{779} O_{780} O_{781} O_{782} O_{783} O_{784} O_{785} O_{786} O_{787} O_{788} O_{789} O_{790} O_{791} O_{792} O_{793} O_{794} O_{795} O_{796} O_{797} O_{798} O_{799} O_{800} O_{801} O_{802} O_{803} O_{804} O_{805} O_{806} O_{807} O_{808} O_{809} O_{810} O_{811} O_{812} O_{813} O_{814} O_{815} O_{816} O_{817} O_{818} O_{819} O_{820} O_{821} O_{822} O_{823} O_{824} O_{825} O_{826} O_{827} O_{828} O_{829} O_{830} O_{831} O_{832} O_{833} O_{834} O_{835} O_{836} O_{837} O_{838} O_{839} O_{840} O_{841} O_{842} O_{843} O_{844} O_{845} O_{846} O_{847} O_{848} O_{849} O_{850} O_{851} O_{852} O_{853} O_{854} O_{855} O_{856} O_{857} O_{858} O_{859} O_{860} O_{861} O_{862} O_{863} O_{864} O_{865} O_{866} O_{867} O_{868} O_{869} O_{870} O_{871} O_{872} O_{873} O_{874} O_{875} O_{876} O_{877} O_{878} O_{879} O_{880} O_{881} O_{882} O_{883} O_{884} O_{885} O_{886} O_{887} O_{888} O_{889} O_{890} O_{891} O_{892} O_{893} O_{894} O_{895} O_{896} O_{897} O_{898} O_{899} O_{900} O_{901} O_{902} O_{903} O_{904} O_{905} O_{906} O_{907} O_{908} O_{909} O_{910} O_{911} O_{912} O_{913} O_{914} O_{915} O_{916} O_{917} O_{918} O_{919} O_{920} O_{921} O_{922} O_{923} O_{924} O_{925} O_{926} O_{927} O_{928} O_{929} O_{930} O_{931} O_{932} O_{933} O_{934} O_{935} O_{936} O_{937} O_{938} O_{939} O_{940} O_{941} O_{942} O_{943} O_{944} O_{945} O_{946} O_{947} O_{948} O_{949} O_{950} O_{951} O_{952} O_{953} O_{954} O_{955} O_{956} O_{957} O_{958} O_{959} O_{960} O_{961} O_{962} O_{963} O_{964} O_{965} O_{966} O_{967} O_{968} O_{969} O_{970} O_{971} O_{972} O_{973} O_{974} O_{975} O_{976} O_{977} O_{978} O_{979} O_{980} O_{981} O_{982} O_{983} O_{984} O_{985} O_{986} O_{987} O_{988} O_{989} O_{990} O_{991} O_{992} O_{993} O_{994} O_{995} O_{996} O_{997} O_{998} O_{999} O_{1000} O_{1001} O_{1002} O_{1003} O_{1004} O_{1005} O_{1006} O_{1007} O_{1008} O_{1009} O_{1010} O_{1011} O_{1012} O_{1013} O_{1014} O_{1015} O_{1016} O_{1017} O_{1018} O_{1019} O_{1020} O_{1021} O_{1022} O_{1023} O_{1024} O_{1025} O_{1026} O_{1027} O_{1028} O_{1029} O_{1030} O_{1031} O_{1032} O_{1033} O_{1034} O_{1035} O_{1036} O_{1037} O_{1038} O_{1039} O_{1040} O_{1041} O_{1042} O_{1043} O_{1044} O_{1045} O_{1046} O_{1047} O_{1048} O_{1049} O_{1050} O_{1051} O_{1052} O_{1053} O_{1054} O_{1055} O_{1056} O_{1057} O_{1058} O_{1059} O_{1060} O_{1061} O_{1062} O_{1063} O_{1064} O_{1065} O_{1066} O_{1067} O_{1068} O_{1069} O_{1070} O_{1071} O_{1072} O_{1073} O_{1074} O_{1075} O_{1076} O_{1077} O_{1078} O_{1079} O_{1080} O_{1081} O_{1082} O_{1083} O_{1084} O_{1085} O_{1086} O_{1087} O_{1088} O_{1089} O_{1090} O_{1091} O_{1092} O_{1093} O_{1094} O_{1095} O_{1096} O_{1097} O_{1098} O_{1099} O_{1100} O_{1101} O_{1102} O_{1103} O_{1104} O_{1105} O_{1106} O_{1107} O_{1108} O_{1109} O_{1110} O_{1111} O_{1112} O_{1113} O_{1114} O_{1115} O_{1116} O_{1117} O_{1118} O_{1119} O_{1120} O_{1121} O_{1122} O_{1123} O_{1124} O_{1125} O_{11

Simplified Representation :



x_t

$i = row$
 $t = timestep$



→ Types of RNN

1. One to Many
2. Many to One
3. Many to Many

(One to One is simple ANN)

1. One to Many :

• single input , sequence of outputs.

eg. Music Generation, Image captioning

2. Many to One :

• sequence of inputs , single output.

eg. sentiment Analysis , Spam detection

3. Many to Many : (sub types : Fixed Length)
Variable Length)

• sequence of inputs , sequence of outputs

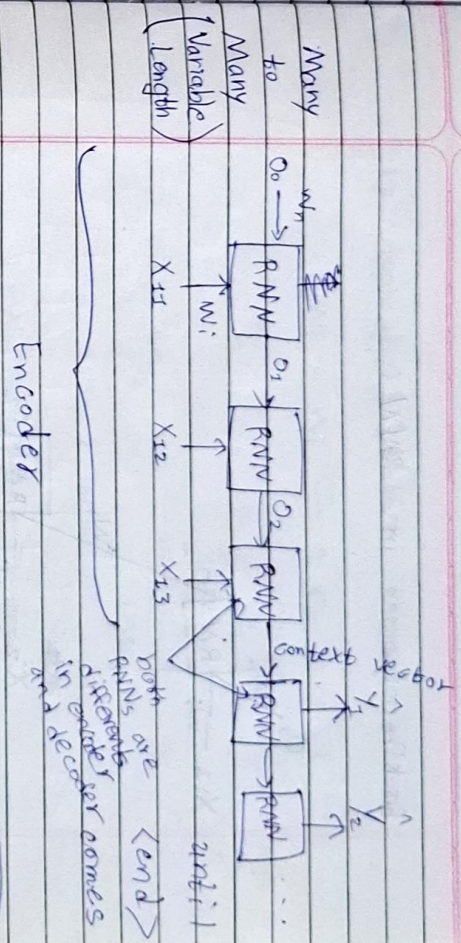
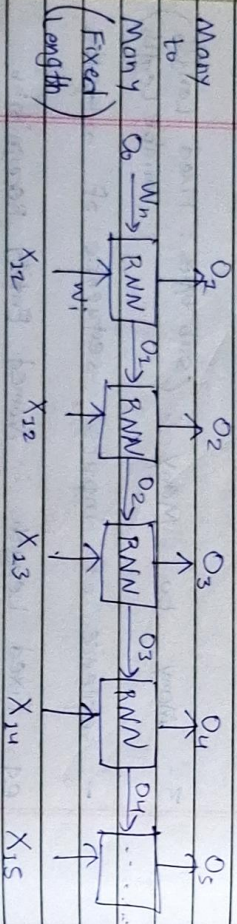
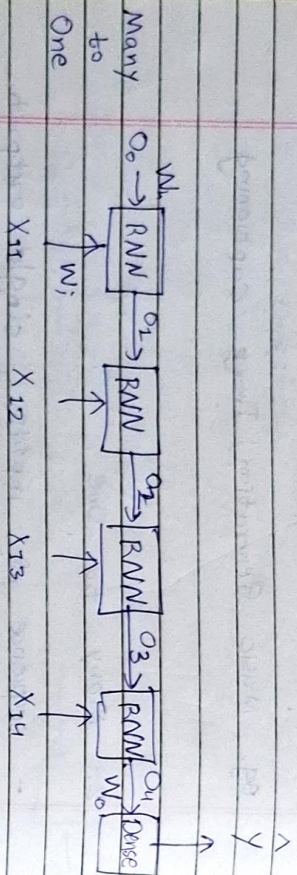
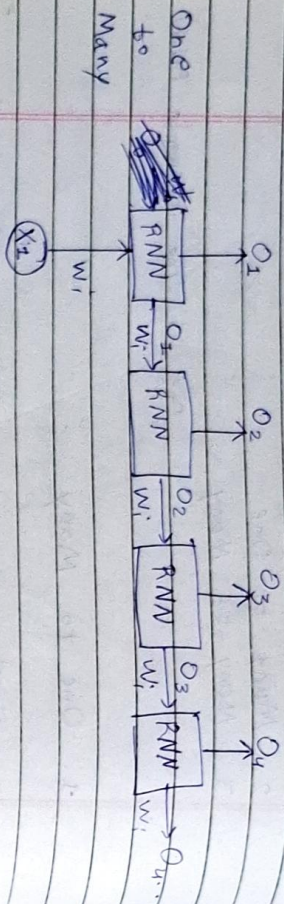
eg. Fixed Length : Named Entity Recognition

Variable Length : ~~Text~~ language Translation,

Video captioning,

Text Summarization

Structures of All types :

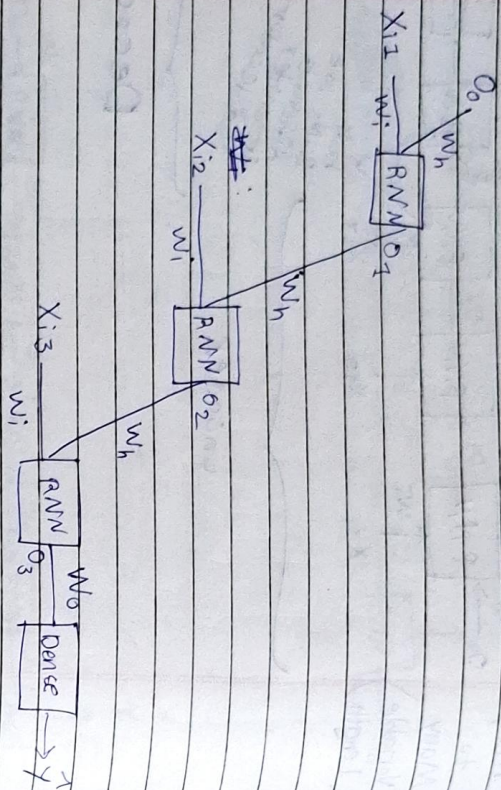


Encoder

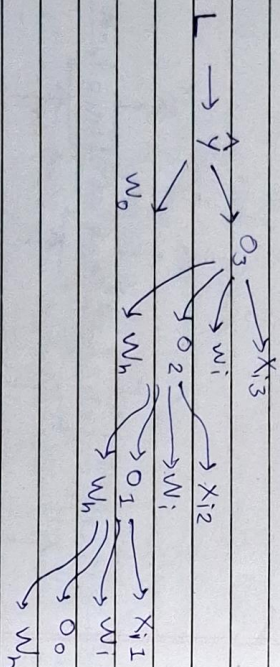
Decoder

until
different
in decoder

Back Propagation in RNN:



Compute Loss,



by Applying Gradient Descent:

$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1}$$

$$w_h = w_h - \eta \frac{\partial L}{\partial w_h}$$

$$w_0 = w_0 - \eta \frac{\partial L}{\partial w_0}$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial O_3} \frac{\partial O_3}{\partial w_1} + \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial O_2} \frac{\partial O_2}{\partial O_3} \frac{\partial O_3}{\partial w_1}$$

$$+ \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial O_1} \frac{\partial O_1}{\partial O_2} \frac{\partial O_2}{\partial O_3} \frac{\partial O_3}{\partial w_1}$$

$$= \sum_{j=1}^S \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial O_j} \frac{\partial O_j}{\partial w_1}$$

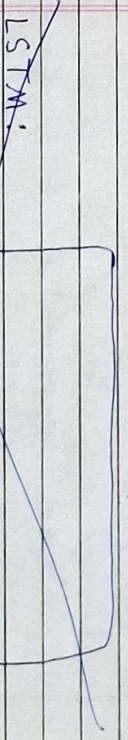
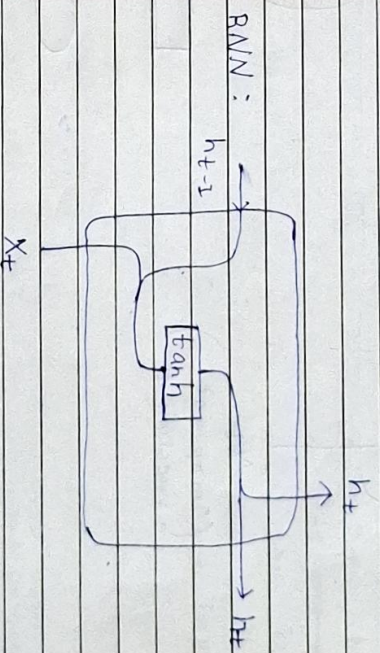
$$\frac{\partial L}{\partial w_h} = \sum_{j=1}^n \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial O_j} \frac{\partial O_j}{\partial w_h}$$

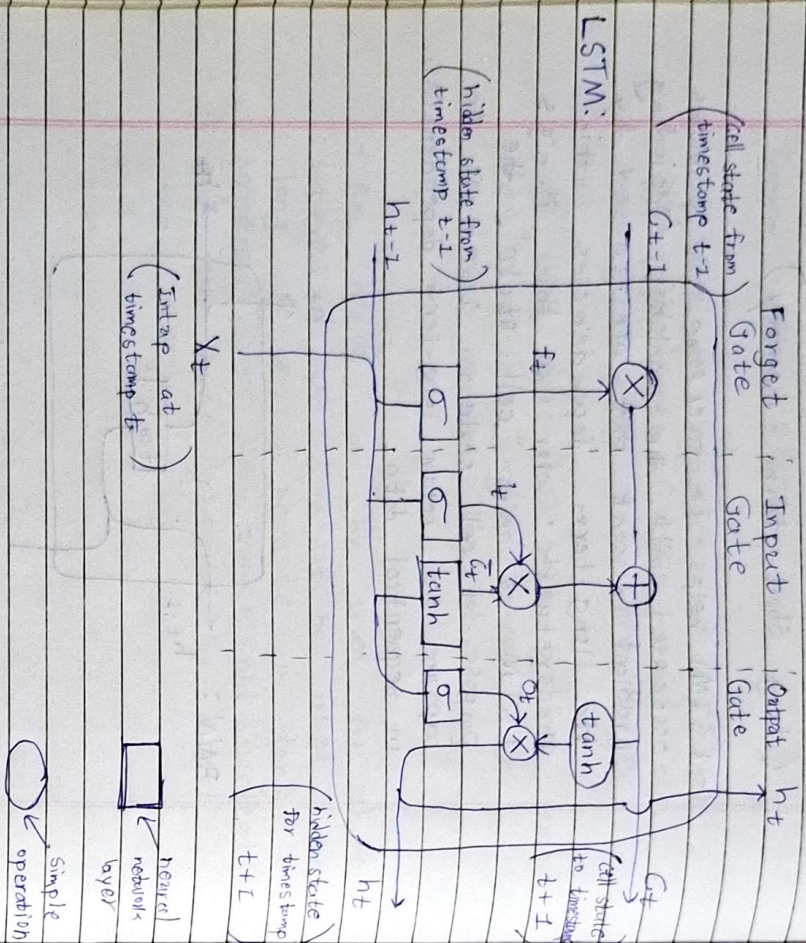
$$\frac{\partial L}{\partial w_0} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_0}$$

LSTM

(Long Short-term memory)

- LSTM helps to overcome the problems associated with the vanishing/exploding gradient descent that are caused by the long-term dependencies within the network. For all this things LSTM is using "cell state". the purpose of cell state in LSTM is to capture and retain long-term dependencies in sequential data;





Hidden state (h_t) (short-term memory):

- hidden state is used for representation of the previous inputs that are used for prediction. So output of the model is dependent on hidden state.

Cell state (C_t) (long-term memory):

- cell state is used for capture and retain long-term dependencies, by selectively storing, forgetting, and adding information based on previous and current information, which is visible in forget gate and input gates.

Forget gate:

- The forget gate, as the name suggests, decides what information should be forgotten. A sigmoid layer is used to make this decision. This sigmoid layer is called the "forget gate layer".

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate :

- The input gate gives new information to the LSTM and decides if that new information is going to be stored in the cell state.

$$i_t = \sigma(w_i [h_{t-1}, x_t] + b_i)$$

$$\bar{c}_t = \tanh(w_c [h_{t-1}, x_t] + b_c)$$

- Input gate has 3 parts :

1. A sigmoid layer decides how much to be updated. This layer is called the "input gate layer".

2. A tanh layer creates a vector of new candidate values, $\bar{c}_t$, that could be added to the state

3. Then we combine these 2, $i_t * \bar{c}_t$, and update the cell state.

The new cell state c_t is obtained by adding the output from forget and input gates.

$$c_t = c_{t-1} * f_t + i_t * \bar{c}_t$$

Output gate :

- The output of the LSTM unit depends on the new cell state.

- First a sigmoid layer decides what parts of the cell state we're going to output. Then, a tanh layer is used on the cell state to squash the values between -1 and 1, which is finally multiplied by the sigmoid gate output.

$$o_t = \sigma(w_o [h_t, x_t] + b_o)$$

$$h_t = o_t * \tanh(c_t)$$

- As mentioned in the article, LSTMs can hold information longer by forgetting and remembering information. This is achieved by 4 components - a cell state and 3 gates. It also combats the vanishing gradient problem, which was a limitation with RNNs. This gives LSTMs an edge over vanilla RNNs.

GRU

(Gated Recurrent Unit)

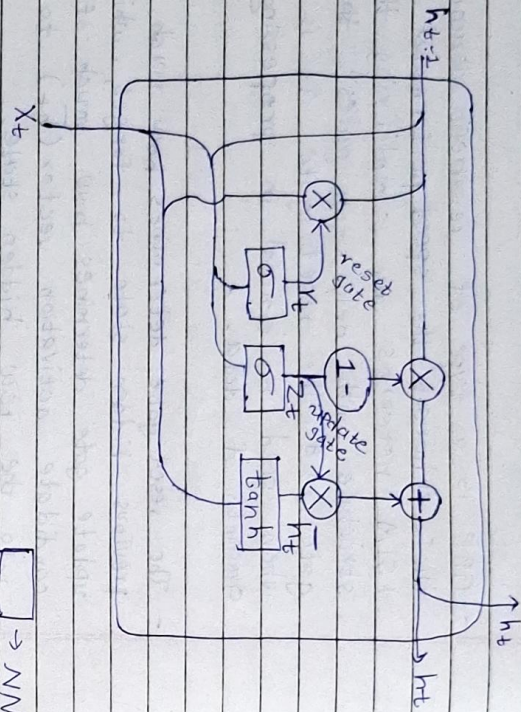
- GRU is a type of recurrent neural network that enhances the speed performance of LSTM networks by simplifying the structure with only two gates: the update gate and the reset gate. It is used when speed is crucial in processing large amounts of data.

- The reset gate determines how much of the previous hidden state to forget, while the update gate determines how much of the candidate activation vector (h_t) to incorporate into the new hidden state.

- GRU is a popular alternative to LSTM for modelling sequential data, especially in cases where computational resources are limited or where a simpler architecture is desired.

- candidate activation vector is used to update the hidden state for the next time step. it is computed using two gates: the reset gate and the update gate. The reset gate determines how much of the previous hidden state to forget, while the update gate determines how much of the candidate activation vector to incorporate into the new hidden state.

GRU:



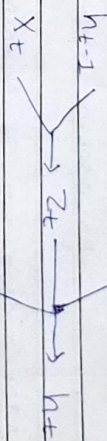
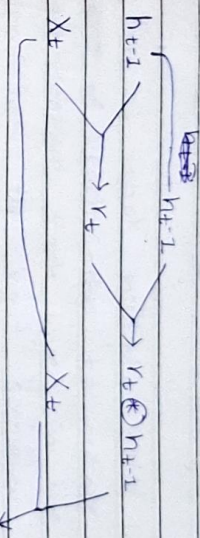
$\rightarrow NN$

$\rightarrow$ simple operation

$[0.2 \ 0.3 \ 0.7]$

$\uparrow$ $\textcircled{1}$

$[0.8 \ 0.7 \ 0.3]$



h_{t-1}

reset gate, $r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$

update gate, $z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$

candidate activation vector,

$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$

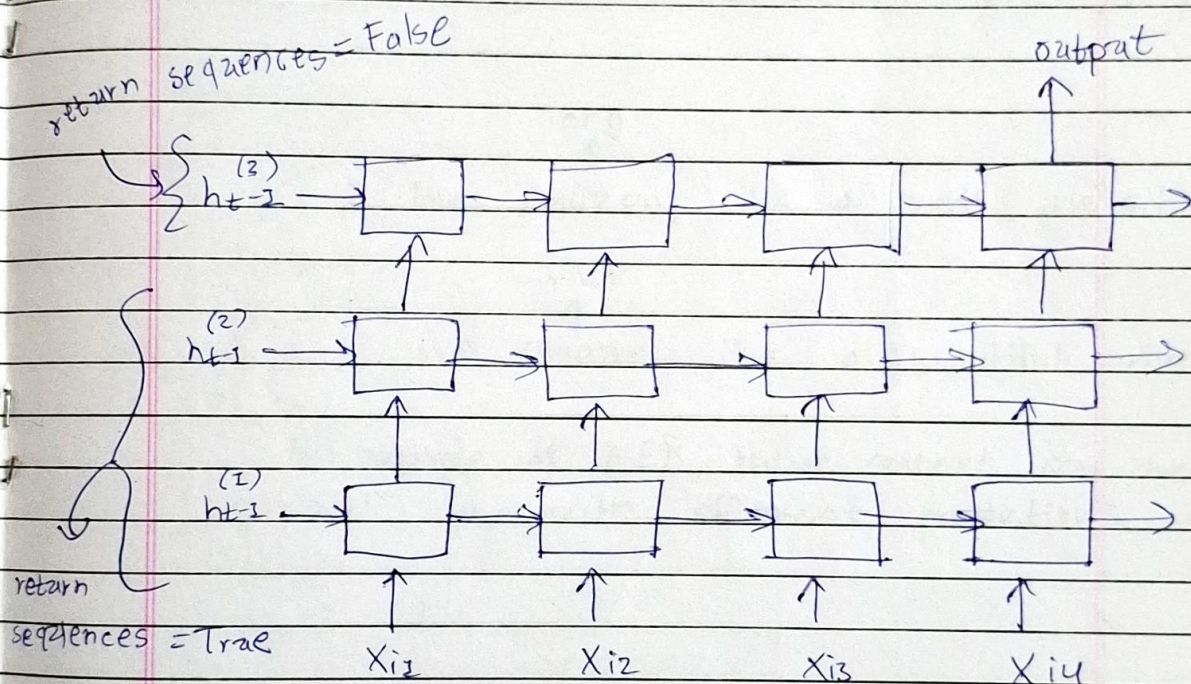
new hidden state,

$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

Deep RNN / LSTM / GRU

(Stacked RNN / LSTM / GRU)

- You can stack multiple RNN layers on top ~~let~~ of each other to create a deep architecture. Each layer in the stack processes the input sequence sequentially, and its hidden state is passed as input to the next layer. The output of the top layer is often used for making predictions or further processing.
- Let's ~~say~~ take example of one such stacked RNN with four inputs and three layers

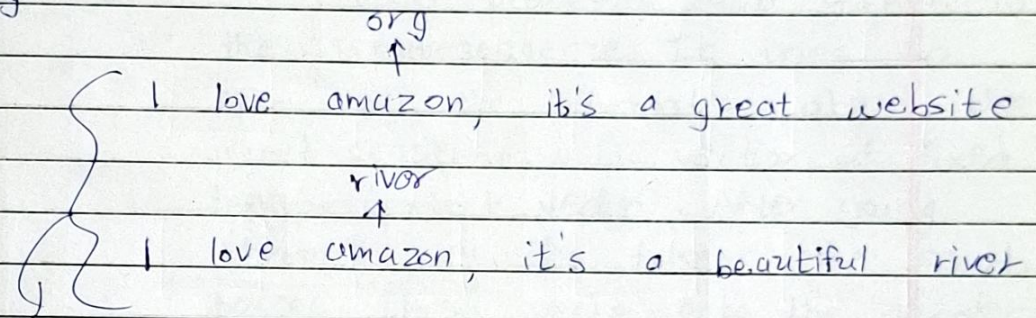


- For deeper understanding please refer campusX deep RNN video, it will help you know how flow is working.

Bidirectional RNN

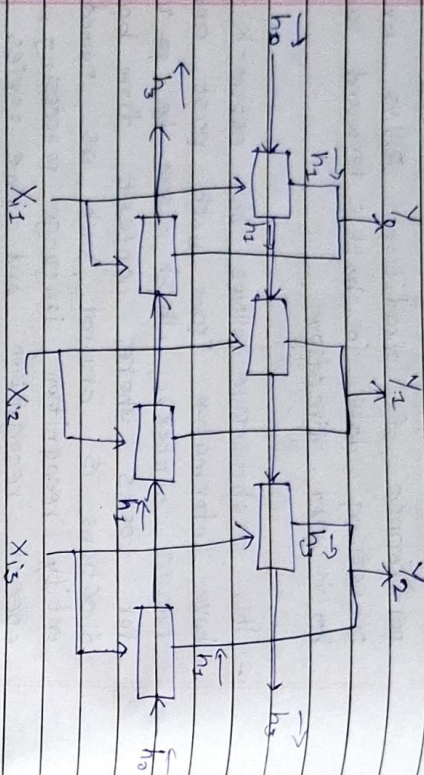
- A Bidirectional RNN is a type of RNN that is designed to improve the performance of traditional RNNs by processing data in both forward and back ward directions
- This architecture allows the network to have information from both past and future contexts, which can be ~~p~~ useful for tasks where context from both directions is crucial such as named entity recognition, language processing, speech recognition, and time series analysis.

eg.



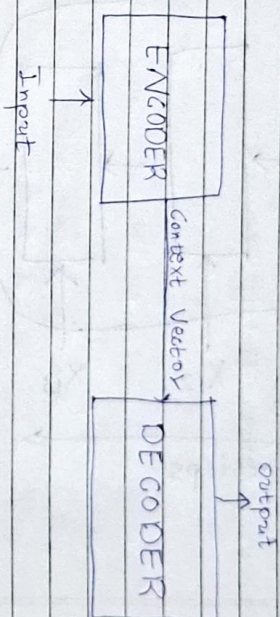
in this example of NER, future context can be helpful to make efficient predictions

Architecture of BiRNN:



Encoder - Decoder

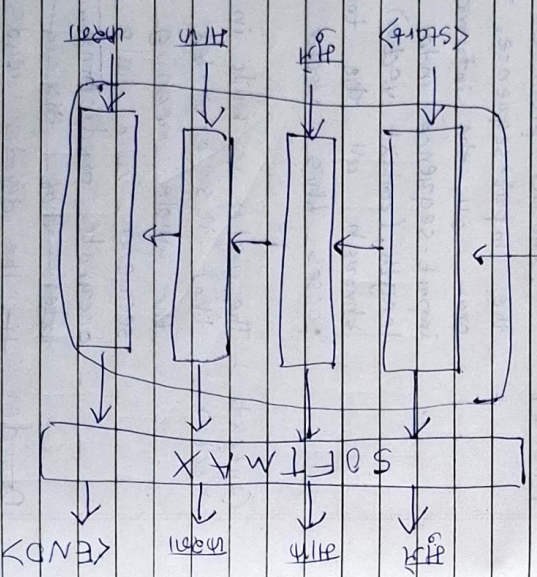
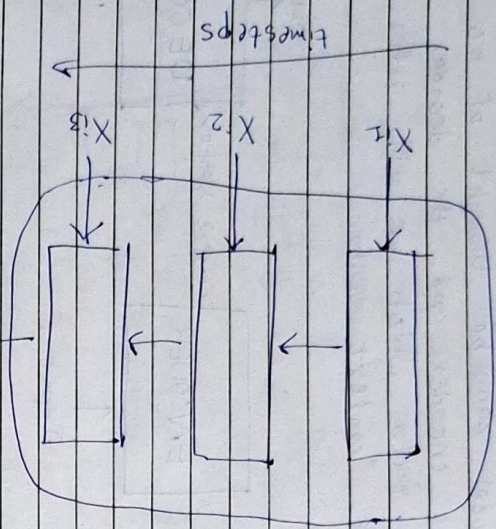
-At a very high level, an encoder-decoder model can be thought of as two blocks, the encoder and the decoder connected by a vector which we will refer to as the "context vector".



Encoder : The encoder processes each token in the input-sequence. It tries to cram all the information about the input-sequence into vector of fixed length, (context vector). After going through all the tokens, the encoder passes this vector onto the decoder.

Context : The vector is built in such a way that it's expected to encapsulate the whole meaning of the input sequence and help the decoder make accurate predictions. We will see later that this is the final

Decoder : The decoder reads the context vector and tries to predict target-sequence token by token.



- For training and prediction of encoder-decoder is given in depth in campusX video and medium article, so go and refer to that.

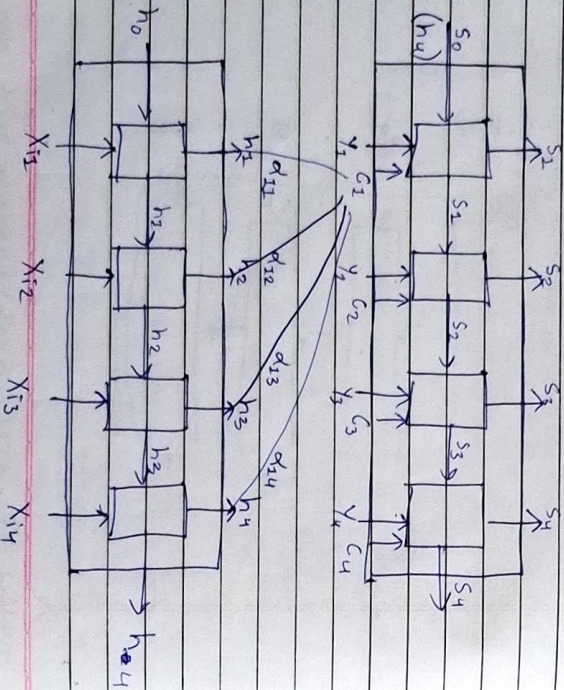
Limitation: large sentence because of fixed-length context vector

- We can take stacked/deep LSTM for better predictions

Attention

- when a large sentence is processed by an encoder capturing its complete context within fixed-length context vector proves to be a difficult task. This limitation can significantly affect translation accuracy.

- The attention mechanism enables models to selectively focus on relevant parts of the input sequence while generating each element of the output sequence, improving accuracy and capturing long-range dependencies in tasks like machine translation.



For decoder we are giving another dynamic vector which will focus on ~~more~~ important hidden cells of encoder ~~that~~, that vector is c_i

attention input

Vanilla encoder decoder encoder decoder
attention

For i^{th} LSTM of decoder,

$$\begin{bmatrix} y_i, s_{i-1} \end{bmatrix} \quad \left\{ \begin{array}{l} [y_i, s_{i-1}, c_i] \\ \uparrow \\ \text{attention input} \end{array} \right.$$

embedding output size

$$c_t = \sum_{i=1}^N \alpha_{ti} * h_i$$

in above example for timestamp $t=2$,

$$c_2 = \alpha_{21}h_1 + \alpha_{22}h_2 + \alpha_{23}h_3 + \alpha_{24}h_4$$

α_{ij} represents attention weight for s_{i-1} and h_j .

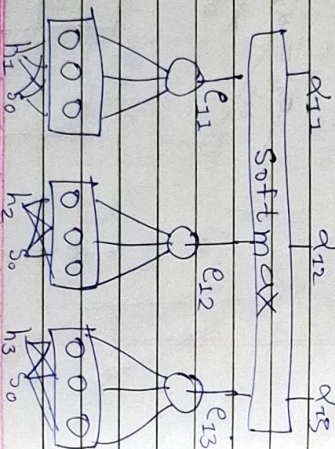
- The attention weights (α_{ij}) are determined by considering two main factors: the current hidden state of the encoder (h_j) at the corresponding timestamp and the previous hidden state value of the decoder (s_{i-1}). So, we have to find such function f which will ~~generate~~ calculate α_{ij} from given input s_{i-1} and h_j .

$$\alpha_{ij} = f(h_j, s_{i-1})$$

$$\text{eg. } d_{i3} = f(h_3, s_i)$$

- We can put ANNs to devise this equation, enabling the model to dynamically determine the attention weights during the decoding process.

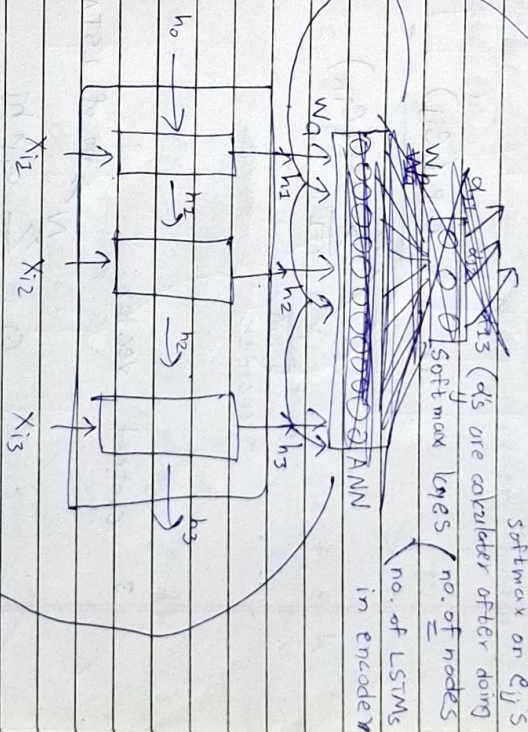
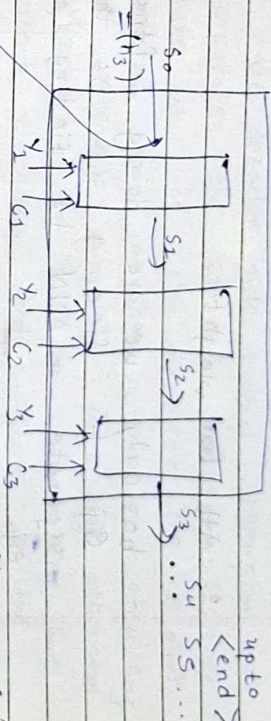
- In attention mechanisms, two popular methods for calculating attention weight values are Bahdanau attention and Luong attention.



This is the right structure.

Bahdanau Attention:

- It is using ANNs to compute the compatibility score,



there are separate

ANN's for finding each

α_{ij} however I have

All ANN's with same weight made mistake by showing wrong structure

1. Compatibility score calculation:

$$e_{ij} = \tanh(W_b(W_a \cdot [s_{i-1}, h_i^T]))$$

2. Attention weights:

- basically we are doing softmax on e_{ij} for finding d_{ij} . As there are separate ANN for finding each e_{ij} .

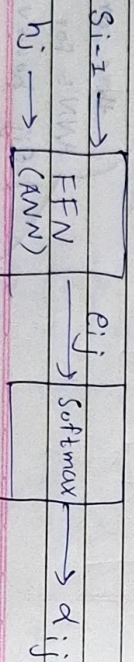
$$d_{ij} = \exp(e_{ij})$$

$$\sum_{k=1}^N \exp(e_{ik})$$

no. of LSTM in encoder

3. context vector:

$$c_i = \sum_{j=1}^N d_{ij} * h_j$$



Luong Attention:

- In Luong Attention, the first difference from Bahdanau attention is that instead of considering the previous hidden state value of the decoder side, Luong attention incorporates the current timestamp hidden state value of the decoder. The second difference lies in the method of calculating the attention weight (α): Luong attention utilizes the dot product between the current timestamp hidden state of the decoder (s_i) and each encoder hidden state (h_i).

$$d_{ij} = F(s_i, h_j) = s_i^T h_j$$

$$\text{if } s_i = [a \ b \ c \ d], \quad h_j = [e \ f \ g \ h]$$

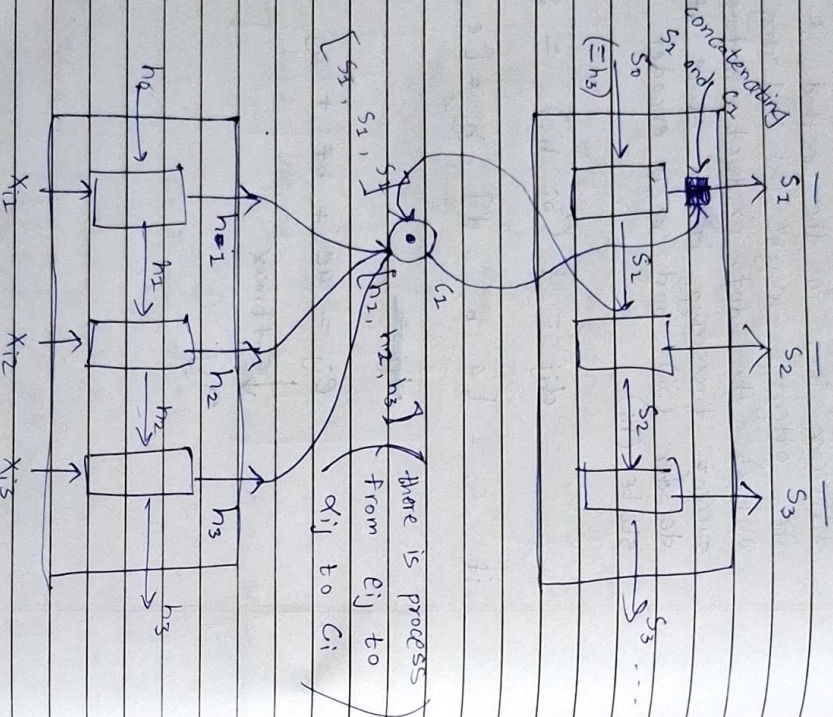
$$d_{ij} =$$

$$e_{ij} = ae + bf + cg + dh$$

↓ softmax

$$d_{ij}$$

- In Long attention, the final difference is that the context vector C_i is passed directly into the output of the LSTM cell, which contrasts with the approach in Bahdanau attention where the context vector is concatenated with the input to the LSTM cell.



in above example C_1 will be calculated with,

$$e_{11} = s_1^T h_1, \quad e_{12} = s_1^T h_2, \quad e_{13} = s_1^T h_3$$

Softmax on e_{11}, e_{12}, e_{13} find

$$a_{11}, a_{12}, a_{13}$$

$$a_{11} = \frac{\exp(e_{11})}{\sum_{i=1}^3 \exp(e_{1i})}$$

⋮

then, $C_1 = \sum_{i=1}^3 a_{1i} * h_i$

- So we are impacting directly and only on the output of each LSTMs.

- far better understanding visit
Campus X video and medium articles.